

UNIVERSIDADE FEDERAL DO CARIRI – UFCA
UNIVERSIDADE ABERTA DO BRASIL – UAB
CENTRO DE EDUCAÇÃO A DISTÂNCIA (CEAD – UFCA)
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (ADS)
PROJETO INTEGRADO III [ADS0038]
PROF. DR. LUÍS FABRÍCIO DE FREITAS SOUZA

ENTREGÁVEL PARCIAL 2 (EP2) - RELATÓRIO

BEATRIZ BENIGNO DE VASCONCELOS - MAT. 2025013076

ALDEMIR FERREIRA DA SILVA JÚNIOR - MAT. 2025012892

FRANCISCO SAVIO SOUSA DA CUNHA - MAT. 2025013352

FRANCISCO DIOGO DE SOUSA SILVA - MAT. 2025013307

JOÃO PAULO LIMA DAVID - MAT. 2025013441

I - Documentação Arquitetural - Escola360 MVP Web

1. Visão Geral da Arquitetura

1.1 Problema que o Sistema Resolve

Atualmente, muitos processos escolares ainda são realizados de forma manual ou através de sistemas desconectados, dificultando:

- O acompanhamento do desempenho dos alunos por pais e responsáveis;
- O gerenciamento eficiente de notas e frequências pelos professores;
- A geração de relatórios e dashboards para gestores;
- A comunicação entre escola, alunos e responsáveis;

O Escola360 busca solucionar essa falta de integração e visibilidade das informações acadêmicas em instituições de ensino ao centralizar em uma única plataforma o registro de notas, frequência e comunicação escolar, garantindo aos alunos, responsáveis, professores e gestores um acompanhamento mais eficiente do desempenho e da rotina escolar.

1.2 Objetivo do MVP

O MVP (Minimum Viable Product) do Escola360 tem como objetivo entregar um sistema web funcional que permita:

- Autenticação segura de diferentes perfis de usuários;
- Gestão acadêmica completa (cadastro de alunos, professores, disciplinas, turmas);
- Lançamento e consulta de notas em tempo real;
- Registro e acompanhamento de frequência;
- Geração de relatórios para tomada de decisão;
- Portal diferenciado para cada tipo de usuário (Gestor, Professor, Aluno, Responsável);

1.3 Usuários do Escola360

O Escola360 é projetado para atender às necessidades informacionais de quatro perfis principais de usuários: Gestores, Professores, Alunos e Responsáveis.

<i>Perfil</i>	<i>Função</i>	<i>Acesso a</i>
Gestor	Administração geral	Todos os módulos, relatórios, dashboards macro
Professor	Gestão acadêmica	Turmas, notas, frequência das disciplinas que leciona
Aluno	Consulta acadêmica	Boletim, frequência, avisos pessoais
Responsável	Acompanhamento	Boletim e frequência dos alunos sob sua tutela

2. Modelo Arquitetural do Sistema

2.1 Estrutura em Camadas

A arquitetura adotada baseia-se no modelo Cliente-Servidor e divide-se em três camadas principais:

- Camada de apresentação (frontend) : Interface web responsiva (html / css / javascript);
- Camada de aplicação: API construída em Python, regras de negócio (python) e ORM (SQLAlchemy).
- Camada de dados: Camada de persistência relacional (PostgreSQL).

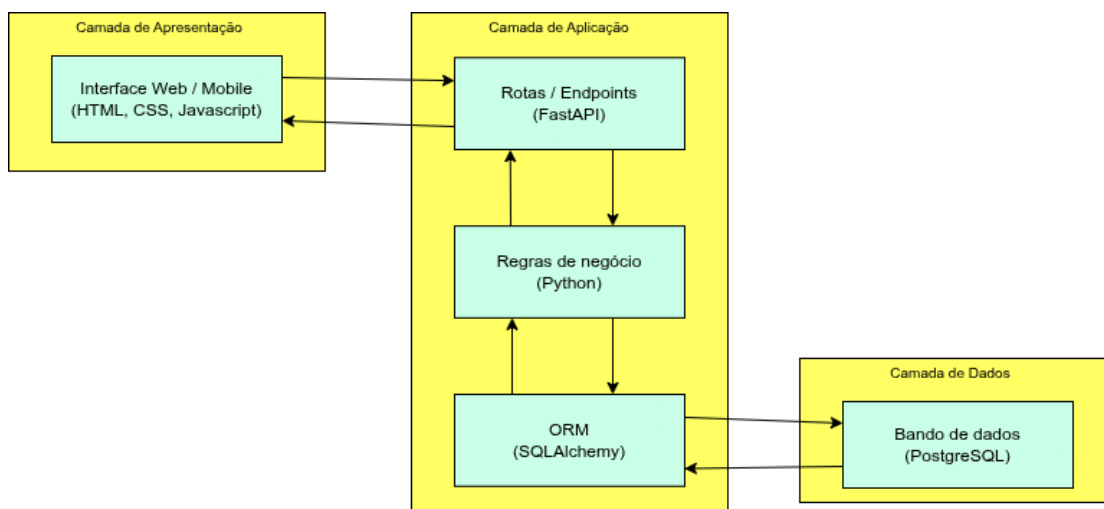


Fig. 01 - Diagrama de Arquitetura do Escola360

2.2 Componentes da Aplicação

2.2.1 Módulo de Autenticação Multifator (MFA)

- **Função:** Garantir a segurança do acesso ao sistema através de duas etapas de verificação. Primeiro, o usuário insere seu e-mail e senha cadastrados. Se estiverem corretos, o sistema exige uma segunda confirmação (como um código enviado por e-mail, SMS ou aplicativo de autenticação).
- **Comunicação:** É a "porta de entrada" obrigatória do sistema. Ele conversa diretamente com o cadastro de **Usuários** para checar as credenciais e, após a dupla validação, libera o acesso para os painéis correspondentes (Gestor, Professor, Aluno ou Responsável).
- **Importância:** Protege os dados sensíveis da escola. Ele impede que pessoas não autorizadas acessem o sistema para alterar notas ou visualizar informações privadas de alunos e responsáveis, mesmo que descubram a senha de alguém.

2.2.2 Cadastro e Controle de Usuários (usuarios.py)

- **Função:** Organizar quem é quem dentro do sistema, dividindo os acessos e permissões de acordo com o perfil de cada um, por exemplo: **Gestores** (administram a escola), **Professores** (lançam notas e faltas) e **Responsáveis** (acompanham os alunos).
- **Comunicação:** Recebe os comandos do módulo de Autenticação e envia as informações necessárias para os componentes de Alunos e Disciplinas.
- **Importância:** Garante que cada usuário veja apenas o que lhe é permitido. Um aluno não pode ver o painel do gestor, e um professor só gerencia as turmas em que leciona.

2.2.3 Painel Administrativo (Dashboards Personalizados por Perfil)

- **Função:** Através da construção e implementação de um sistema próprio de Dashboard, o mesmo tem a função de apresentar uma visão centralizada, gráfica e intuitiva das informações do sistema. Em vez de uma tela única para todos, este componente se transforma com base em quem está logado:

- **Dashboard do Gestor:** Exibe indicadores macros da escola, como taxa de evasão, gráficos de desempenho geral das turmas, relatórios financeiros e ferramentas para gerenciar usuários e disciplinas.
 - **Dashboard do Professor:** Focado na rotina de sala de aula, exibindo atalhos rápidos para suas turmas, calendário de aulas do dia, e painéis fáceis para lançamento de notas e registro de chamadas.
 - **Dashboard do Aluno:** Mostra suas notas, mural de avisos da escola, calendário de provas e o gráfico de sua própria frequência.
 - **Dashboard do Responsável:** Apresenta o acompanhamento pedagógico dos filhos sob sua tutela, notificações de faltas em tempo real e canais de comunicação com a escola.
-
- **Comunicação:** Consome dados de todos os outros módulos (Usuários, Alunos, Disciplinas e Avaliações) para montar os gráficos e tabelas exibidos na tela.

2.2.4 APIs (Possíveis Usos)

2.2.4.1 - APIs de Segurança e Autenticação Multifator (MFA)

- **POST /api/v1/auth/login - API de Validação de Email**
 - **POST /api/v1/auth/mfa-verify - API de Geração de Token**
-
- **Função:** Controlar o fluxo de acesso ao sistema em duas etapas. A primeira valida a combinação de e-mail e senha cadastrados no componente **Usuario**; a segunda gera, envia e valida um código dinâmico temporário.
 - **Como se comunicam:** Recebem as tentativas de login vindas da tela de entrada do usuário. Elas consultam o banco de dados PostgreSQL para verificar se a senha bate. Em caso positivo, comunicam-se com um serviço de disparos (e-mail/SMS) para enviar o código e, após a validação final, emitem um token de acesso seguro.

- **Importância:** São fundamentais para a governança e proteção do sistema. Elas garantem que dados extremamente sensíveis (como registros de menores de idade, telefones de responsáveis e histórico de notas) fiquem completamente blindados contra acessos não autorizados.

2.2.4.2 - APIs de Gestão e Administração (Dashboard do Gestor)

- **SendGrid API - API de disparo de E-mails**

- **Função:** Uma API pronta e especializada em disparar e-mails em massa. O gestor poderia usá-la para enviar os boletins em PDF para todos os responsáveis da escola com um único clique.
- **Como se comunicam:** A comunicação é baseada em modelos de e-mail (*templates*) pré-configurados e requisições assíncronas:
 1. O gestor entra no **Dashboard do Gestor** e clica no botão relacionado ao disparo de boletins
 2. O sistema ativa o componente **relatorios.py**, gera os boletins e faz uma requisição para a API do SendGrid contendo a lista de e-mails dos responsáveis e os arquivos anexados.
 3. O SendGrid recebe esse lote de dados e gerencia a entrega de centenas de e-mails simultaneamente, cuidando para que nenhum deles caia na caixa de spam.
 4. A API do SendGrid devolve um relatório para o Escola360 informando quais pais abriram o e-mail e quais endereços estavam errados.
- **Importância:** Garante formalidade e escalabilidade. Enviar centenas de e-mails diretamente pelo seu servidor local em Python faria o sistema travar ou ser bloqueado por provedores (como Gmail e Outlook). O SendGrid assume essa carga, garantindo que os documentos oficiais cheguem com segurança, além de fornecer relatórios de leitura cruciais para a auditoria do gestor.

2.2.4.3 - APIs de Comunicação

- **WhatsApp Business API - API de Envio de Mensagens Automáticas**

- **Função:** Direcionada preferencialmente aos responsáveis, o sistema iria usar a API oficial do WhatsApp para enviar notificações automáticas para os pais (ex: *"Atenção: Seu filho faltou à aula de Matemática hoje"*).
- **Como se comunicam:** A comunicação aconteceria por meio de gatilhos (*webhooks*) disparados pelas ações do sistema:
 1. O professor entra no **Dashboard do Professor** e registra uma falta para o aluno em **avaliacao.py**.
 2. O backend do Escola360 identifica a falta e prepara uma requisição de texto (ex: *"Olá, informamos que o aluno(a) Ana Pereira faltou à aula de Matemática hoje"*).
 3. O seu sistema faz uma chamada HTTP segura para os servidores da API do WhatsApp, enviando essa mensagem junto com o número do telefone do responsável (obtido em **usuarios.py**).
 4. A API do WhatsApp processa o pedido e entrega a mensagem na tela do celular do pai, mãe ou responsável em segundos.
- **Importância:** Garante imediatismo e maior aproximação com responsáveis, devido principalmente ao fato de que as pessoas visualizam o WhatsApp muito mais rápido do que e-mails ou aplicativos escolares próprios. No funcionamento do sistema, essa API é vital para combater a infrequência escolar e aproximar os pais da rotina dos alunos em tempo real.

2.2.4.4 - APIs de Armazenamento em Nuvem

- **Amazon S3 API (AWS)**

- **Função:** Guardar PDFs de boletins, fotos de documentos de matrícula, históricos escolares e relatórios diretamente no banco de dados PostgreSQL

pode deixá-lo lento e pesado. Sendo assim utilizar uma API dedicada ao armazenamento de informações é uma alternativa muito pragmática

- **Como se comunicam:** Quando o componente **relatorios.py** cria um relatório ou quando um aluno faz o upload de um documento de matrícula, o Escola360 envia o arquivo pesado diretamente para a API de nuvem da Amazon/Google. A API armazena o arquivo com segurança e devolve para o sistema apenas um link web criptografado de acesso rápido.
- **Importância:** Garante alto desempenho e escalabilidade ao sistema. O banco PostgreSQL fica leve (guardando apenas texto e links), e os arquivos ficam protegidos em servidores globais contra perdas ou falhas físicas.

2.2.4.5 - APIs de IA e Análise de Dados (Para Prevenção de Evasão)

- Google Gemini API

- **Função:** Auxiliar na análise detalhada do desempenho escolar dos alunos, propondo melhorias no ensino ou prevendo situações de reprovação ou evasão escolar.
- **Como se comunicam:** O *Dashboard do Gestor* envia um resumo anônimo dos dados de frequência (calculados no Para fechar com chave de ouro o documento técnico do seu **Modelo Arquitetural**, precisamos detalhar a seção de **Módulos Principais do Projeto**. No contexto do Escola360, os módulos de backend escritos em Python representam a inteligência de negócios do sistema, ou seja, as regras fundamentais que ditam como a escola opera.

2.2.5 Módulos Principais

2.2.5.1 - Módulo do Aluno (aluno.py)

- **Função:** Modelar a entidade do estudante através da classe Aluno (que herda as características básicas de **Usuario**). Este módulo centraliza os métodos de consulta e o portfólio acadêmico individual do discente.

- **Como se comunica:** Recebe dados de desempenho (notas e faltas) enviados diretamente pelas operações do componente Professor. Ele também expõe a função **consultar_nota()**, que é consumida pelo **Dashboard do Aluno** e pelo **Dashboard do Responsável** para exibir as informações em tela.
- **Importância:** É a entidade central de valor do Escola360. Todos os outros módulos e as APIs de terceiros (como o WhatsApp que avisa sobre faltas) trabalham para atualizar ou ler informações contidas dentro deste módulo.

2.2.5.2 - Módulo de Matrizes e Grade Curricular (disciplinas.py)

- **Função:** Gerenciar a estrutura curricular da instituição através da classe **Disciplina**. Ele é responsável por agrupar os alunos matriculados e associar o professor a cada matéria específica (como Matemática ou Português).
- **Como se comunica:** Serve de ponto de convergência. Quando o script de testes (**main.py**) ou o painel visual aciona um lançamento de nota, a **Disciplina** armazena uma referência dessa nota em sua lista interna (**__notas**), permitindo que a matéria conheça o desempenho médio de seus alunos.
- **Importância:** Garante a organização e a segmentação pedagógica da escola. Sem esse módulo, o sistema não conseguiria separar os históricos escolares por matéria, inviabilizando a lógica de boletins e o cálculo de médias por disciplina.

2.2.5.3 - Módulo de Registro Pedagógico (avaliacao.py)

- **Função:** Implementar de forma granular as classes **Nota** e **Frequencia**. Ele lida com os menores fragmentos de informação do dia a dia escolar: o valor numérico de um teste, a data em que ocorreu, o tipo de avaliação (Prova, Trabalho, etc) e o status de presença.
- **Como se comunica:** É instanciado diretamente pelo componente **Professor** dentro do fluxo de aula. Ao ser criado, o objeto **Nota** ou

Frequencia se vincula bidirecionalmente, injetando a si mesmo dentro do **Aluno** correspondente e da **Disciplina** respectiva.

- **Importância:** Atua como o **fiscal das regras de negócio**. Ele barra erros humanos diretamente no backend por meio de validações rígidas (disparando um **ValueError** se uma nota for menor que 0 ou maior que 10, ou se o status da chamada for diferente de "Presença" ou "Falta"). Isso blinda o banco de dados PostgreSQL contra informações inconsistentes.

2.2.5.4 - Módulo de Relatórios e Auditoria (relatorios.py)

- **Função:** Modelar o objeto **Relatorio**, estruturando metadados como ID do relatório, título e a data de geração automática.
- **Como se comunica:** É acionado exclusivamente por usuários autorizados que assinam o contrato da interface **GeradorRelatorio** (como a classe **Gestor**). Ele minera os dados históricos salvos nos módulos de alunos e disciplinas para consolidar as informações de auditoria.
- **Importância:** Fornece a inteligência estratégica que alimenta o Dashboard do Gestor. É o módulo responsável por transformar dados brutos e isolados de notas e faltas em documentos organizados (capazes de serem enviados por e-mail via APIs como a do SendGrid) para que a diretoria tome decisões preventivas contra a evasão escolar.

2.3 Tecnologias Utilizadas

As tecnologias escolhidas para a construção do sistema visam garantir performance, segurança e escalabilidade. Além disso, elas foram escolhidas com base em critérios de robustez, maturidade do ecossistema, facilidade de manutenção e aderência aos requisitos acadêmicos e funcionais do projeto. A seguir listamos as principais tecnologias e suas respectivas funções no sistema:

- **Python 3.10+ (Backend):** Linguagem principal do projeto. Escolhida por sua sintaxe limpa, vasto ecossistema e excelente capacidade de modelagem orientada a objetos, ideal para as regras de negócio escolares.
- **HTML5, CSS3 e JavaScript (Frontend):** Tecnologias base da web. Utilizadas para construir uma interface SPA (Single Page Application) responsiva, garantindo que o sistema funcione bem em desktops e dispositivos móveis sem exigir instalação.
- **PostgreSQL (Banco de Dados):** SGBD relacional robusto. Escolhido por oferecer suporte avançado a integridade referencial, constraints e validações semânticas, garantindo que os relacionamentos complexos (ex: aluno-disciplina-professor) sejam geridos com segurança.
- **SQLAlchemy (ORM):** Framework de mapeamento objeto-relacional para Python. Faz a ponte entre as classes Python e o PostgreSQL, eliminando a necessidade de escrita manual de SQL e prevenindo vulnerabilidades como SQL Injection.
- **Git e GitHub (Versionamento):** Ferramentas padrão de mercado para controle de versão, permitindo trabalho colaborativo, revisão de código e backup seguro do histórico do projeto.

2.4 Integrações do Sistema

O Escola360 foi projetado para se integrar com serviços externos que ampliam suas funcionalidades, especialmente nos aspectos de comunicação, armazenamento e análise de dados. Aqui destacamos:

- **SendGrid API:** Integração planejada para o módulo de relatórios, permitindo o disparo assíncrono e em massa de e-mails (como boletins em PDF e comunicados formais) diretamente do painel do gestor.
- **WhatsApp Business API:** Integração focada na comunicação em tempo real. Será acionada via webhooks sempre que uma falta for registrada no sistema, notificando imediatamente o responsável via mensagem no celular.
- **Amazon S3 (AWS):** Integração de armazenamento em nuvem. Será utilizada para desonerar o banco de dados relacional, guardando arquivos pesados (documentos de matrícula, fotos, PDFs) e devolvendo apenas links de acesso ao sistema.

- **Google Gemini API:** Integração de Inteligência Artificial focada no painel do gestor. Consumirá dados anonimizados de frequência e notas para gerar análises preditivas sobre o risco de evasão escolar.

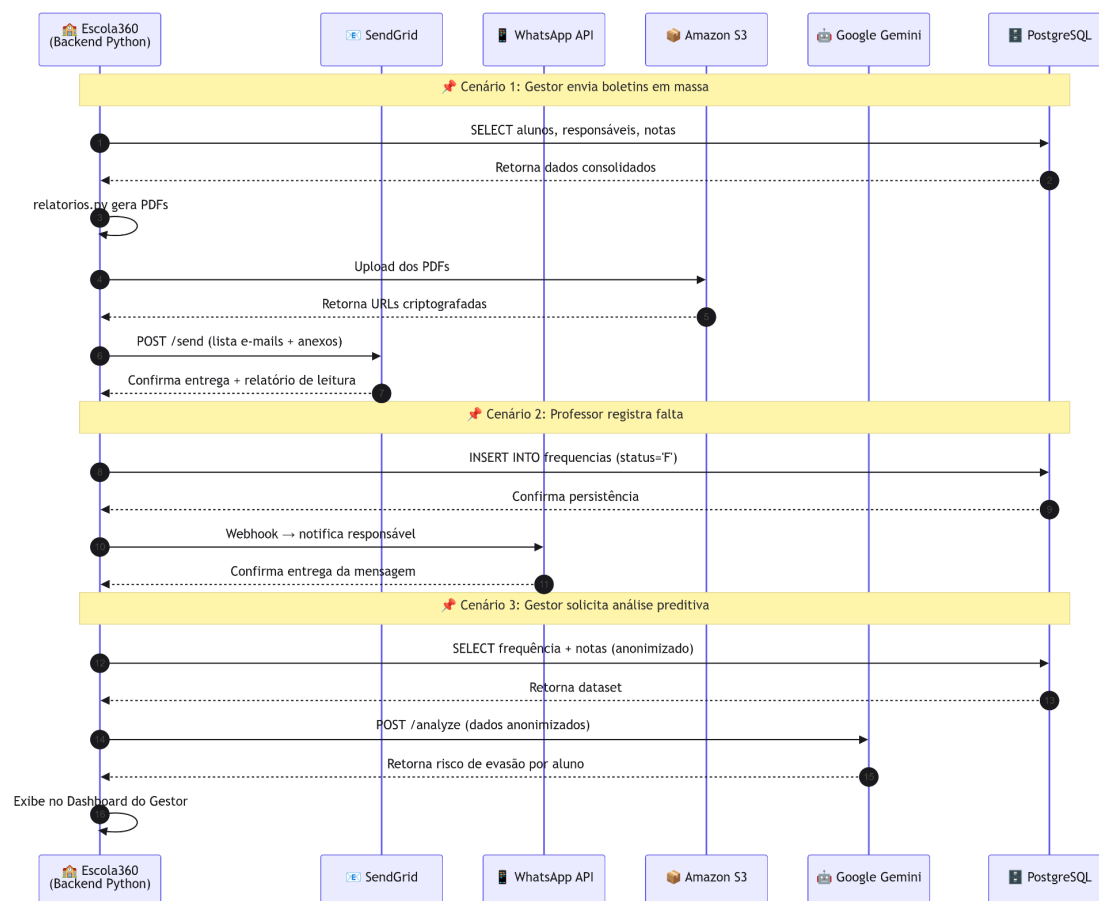


Fig. 02 - Fluxo de comunicação entre o Escola360 e os serviços externos

3. Decisões Arquiteturais

- **Escolha da arquitetura MVC**

A escolha da arquitetura MVC é justificada pelas necessidades do sistema de possuir diversas funcionalidades referentes aos diferentes perfis dentro do sistema Escola360, os perfis de usuários como: Gestores, Professores, Alunos e Responsáveis, possuem funcionalidades específicas e compartilhando outras como autenticação, envios de comunicados e consultas de informações. A arquitetura do MVC por sua vez permitiu que as

telas estivessem responsáveis somente pela interação com o usuário, mantendo assim as regras específicas do negócio e o acesso ao banco de dados separado.

A adoção da arquitetura MVC divide o sistema em camadas definidas onde o View, Controller e Model, são partes com responsabilidades independentes, mas dependentes umas das outras, sua implementação torna o sistema mais organizado dado a gama de funcionalidades e recursos a serem adicionadas. A organização nesse modelo também permite que alterações possam ser feitas em uma camada específica do sistema que se quer alterar, sem que isso altere partes distintas do sistema.

- **Uso de APIs Rest**

O Sistema Escola360 apresenta a API REST que utiliza os principais métodos HTTP padronizados: GET, POST, PUT, DELETE, esses métodos são respectivamente responsáveis por Consultar, Cadastrar, Atualizar e Excluir dados. O sistema utiliza os métodos dessa API de forma a permitir que o usuário possa realizar diferentes processos como login, cadastrar usuários, consultar dados, gerar relatórios, lançamento de frequências ou notas, contudo, dependendo do usuário em questão que esteja autenticado no momento os processos diferem, essa adoção permite também que o sistema seja de fácil manutenção e integração com outros sistemas e aplicações aos quais o sistema seja ligado ou utilizado.

- **Separação frontend/backend:**

Optou-se pelo desacoplamento entre a interface (frontend) e a regra de negócios (backend) para permitir o desenvolvimento paralelo e independente das equipes. Essa decisão traz vantagens como a possibilidade de trocar a tecnologia do frontend no futuro sem afetar o backend, além de tornar a API reutilizável caso a instituição decida lançar um aplicativo mobile nativo no futuro. Isso aumenta drasticamente a escalabilidade e a facilidade de manutenção do código.

- **Organização em microsserviços ou monolito:**

O sistema da Escola360, foi organizado na arquitetura monolítica, onde as funcionalidades apresentadas pela aplicação estão integradas a uma única aplicação, desta maneira, o desenvolvimento e a manutenção do sistema são simplificados para que a complexidade de implantação do sistema seja reduzida. A arquitetura monolítica também facilita o gerenciamento do código e a integração entre os diferentes módulos e funcionalidades da aplicação, diferente da arquitetura em microsserviços que apresenta uma execução independente para cada serviço oferecido.

- **Estratégias de segurança**

O sistema Escola360 possui autenticação como uma das suas principais estratégias de segurança, onde o usuário informa suas credenciais, login e senha, que então serão verificados no banco de dados antes de permitir o acesso a aplicação, para que ao confirmar sua validade, as funcionalidades sejam liberadas. Em seguida, após a autenticação, cada usuário acessa apenas as funcionalidades adequadas para seu perfil na organização de acordo com seu perfil de acesso. Desta forma, os gestores, professores, alunos e responsáveis possuem permissões específicas, o que garante que cada usuário possa acessar e utilizar apenas as funcionalidades autorizadas.

- **Organização do banco de dados.**

O sistema Escola360 utiliza o PostgreSQL como SGBD, adotando um modelo relacional para organizar as informações da aplicação. Os dados são armazenados em tabelas relacionadas, permitindo que informações de gestores, alunos, professores, responsáveis, turmas, notas, frequências e comunicados sejam estruturadas de forma organizada, garantindo a integridade e consistência das informações que permanecem relacionadas entre si, além de evitar a redundância de dados. Por fim, o relacionamento entre as tabelas também contribui para a integridade dos dados e facilita a manutenção do sistema.

4. Uso de Boas Práticas e Padrões Arquiteturais

4.1 Boas Práticas

- **Separação de responsabilidades**

O backend do Escola360 foi organizado com cada arquivo Python responsável por um domínio específico do sistema, como **aluno.py** para a gestão de alunos, **avaliacao.py** para notas e avaliações, **disciplinas.py** para as disciplinas, **relatorios.py** para geração de relatórios e **usuarios.py** para o controle de usuários e perfis. Essa separação facilita a manutenção e a evolução do código ao longo do tempo.

- **Organização de pastas**

A estrutura do projeto foi organizada com os arquivos separados por responsabilidade e a camada de banco de dados isolada na pasta **Database**. Essa organização permite que qualquer membro da equipe localize rapidamente o arquivo relevante para cada funcionalidade, reduzindo o tempo gasto em navegação pelo projeto e evitando conflitos durante o desenvolvimento colaborativo.

- **Versionamento com Git**

O projeto possui repositório criado no GitHub, estabelecendo a base para o controle de versão colaborativo. O versionamento ativo com commits e branches está previsto para as próximas fases do desenvolvimento, à medida que a equipe avança para a implementação do portal web e da API REST.

- **Responsividade**

As telas do Escola360 foram prototipadas no Figma considerando a resolução de notebooks, ambiente principal de uso de gestores, coordenadores e professores ao acessar o painel administrativo. Para os pais e responsáveis, que tendem a acessar o portal pelo celular, a responsividade será implementada na fase de desenvolvimento do portal web, quando o layout será adaptado para diferentes tamanhos de tela por meio de técnicas de CSS responsivo. Isso garantirá que o conteúdo se reorganize automaticamente conforme o dispositivo utilizado, sem perda de legibilidade ou usabilidade, mantendo a experiência do usuário consistente independentemente de como o sistema for acessado.

- **Segurança de autenticação**

O sistema prevê controle de acesso por perfil, garantindo que cada usuário (gestor, professor, pai ou aluno) acesse apenas as informações pertencentes ao seu nível de permissão. A implementação incluirá senhas criptografadas e tokens de autenticação seguros, em conformidade com as diretrizes da LGPD.

4.2 Padrões Arquiteturais

- **Arquitetura em camadas**

O projeto adota uma separação entre a camada de dados representada pela pasta **Database** (com a configuração e conexão ao PostgreSQL) e a camada de lógica de negócio, representada pelos arquivos Python de cada entidade. De forma que cada camada tem responsabilidade bem definida e pode evoluir de forma independente.

- **Cliente-servidor**

A arquitetura prevista para o produto completo segue o modelo cliente-servidor, onde o portal web (cliente) se comunica com o backend Python (servidor), que por sua vez consulta e pega dados no banco PostgreSQL. Esse modelo é ideal, pois garante separação clara entre interface e lógica de negócio, além de facilitar a escalabilidade da aplicação.

- **REST**

A API de comunicação entre o portal web e o backend será estruturada seguindo os princípios REST, com endpoints organizados por recurso, ou seja, cada funcionalidade do sistema terá um endereço próprio, chamado de endpoint, e o tipo de ação será definido pelo verbo utilizado na requisição. Por exemplo, buscar a lista de alunos será feito com GET /alunos, cadastrar um novo aluno com POST /alunos, atualizar os dados de um aluno existente com PUT /alunos e DELETE para exclusões. Esse padrão torna a comunicação entre as partes do sistema organizada e previsível.

- **Componentização**

Na camada de interface, prototipada no Figma, os elementos visuais foram construídos como componentes reutilizáveis (botões, cards, campos de formulário e painéis). Essa abordagem garante consistência visual em todas as telas e facilita a futura implementação do portal web, onde os mesmos componentes poderão ser reaproveitados em diferentes partes do sistema.

- **SPA (Single Page Application)**

O portal web do Escola360 será desenvolvido como uma Single Page Application, modelo no qual a aplicação carrega uma única página e atualiza o

conteúdo dinamicamente sem recarregar o navegador. Isso proporciona uma navegação mais fluida e rápida para o usuário final, especialmente importante para pais que acessam o painel em momentos rápidos do dia.

Considerações Finais

O modelo arquitetural do Escola360 estabelecido permite consolidar a transição entre as regras de negócio abstratas e um sistema computacional seguro e escalável. A adoção do modelo Cliente-Servidor combinado com uma Arquitetura em 3 Camadas, assim como as demais decisões arquiteturais adotadas são adequadas para o estágio atual do projeto, permitindo a possibilidade de evolução para microsserviços conforme o sistema cresce.

II - [Componente Extensionista] No arquivo README do repositório GitHub, criar uma seção intitulada:

“O que é Arquitetura de Software?”

Nesta seção, a equipe deverá explicar, com suas próprias palavras:

- **O que é arquitetura de software;**
- **Qual sua importância no desenvolvimento de sistemas;**
- **Como ela impacta:**
 - **escalabilidade;**
 - **segurança;**
 - **desempenho;**
 - **manutenção;**
 - **evolução do sistema;**
- **Como a arquitetura influencia diretamente a qualidade de um projeto de software.**

Arquitetura de Software

A arquitetura de software pode ser compreendida como a espinha dorsal que estrutura um sistema. Ela define não apenas seus componentes, mas também as relações entre eles e os princípios que governam sua evolução. Assim como um edifício precisa de um projeto estrutural, um sistema de software necessita de um planejamento arquitetônico para garantir seu funcionamento adequado e eficiente.

Por que a Arquitetura é Importante?

A arquitetura permite a modularização do sistema, dividindo-o em partes colaborativas, o que possibilita o desenvolvimento paralelo entre equipes e reduz a sobrecarga cognitiva dos desenvolvedores. Ela também atua como um artefato de comunicação entre arquitetos, desenvolvedores, gerentes e clientes, alinhando expectativas e orientando decisões técnicas e de negócio.

Além disso, as decisões arquiteturais determinam, ainda antes da implementação, as tecnologias que poderão ser utilizadas, como ocorrerá o fluxo de dados entre os componentes e quais trade-offs serão aceitos — como o equilíbrio entre desempenho, escalabilidade e complexidade.

Influência nos Atributos de Qualidade

Um dos aspectos mais relevantes da arquitetura é sua influência direta sobre os atributos de qualidade, que definem como o sistema se comporta:

Atributo	Impacto da Arquitetura
<i>Escalabilidade</i>	A forma como o sistema é particionado determina sua capacidade de crescimento.
<i>Segurança</i>	O isolamento de dados sensíveis e a definição de fronteiras de confiança reduzem a superfície de ataque.
<i>Desempenho</i>	O arranjo dos componentes e os fluxos de comunicação impactam diretamente a velocidade do sistema.
<i>Manutenibilidade</i>	Em um sistema bem projetado, alterações em uma regra de negócio não exigem modificações extensas em outros módulos.
<i>Evolução</i>	Uma boa arquitetura protege as regras de negócio das mudanças tecnológicas, evitando reescritas completas.

Concluindo, a qualidade de um projeto de software está diretamente relacionada à qualidade de sua arquitetura. É ela que determina se o software terá uma vida útil longa e sustentável ou então, se tornará um sistema legado de difícil manutenção.

III - Link do repositório no GitHub

<https://github.com/saviocunha/escola360.git>

IV - Detalhe como cada membro da equipe contribuiu para o desenvolvimento do entregável:

- **Beatriz Benigno:** Contribuiu na edição do relatório, escrevendo as boas práticas e padrões arquiteturais adotados pela equipe na elaboração do futuro MVP.
- **Diogo Sousa:** Contribuiu na produção do relatório, escrevendo o tópico das Decisões Arquiteturais adotadas para elaboração do MVP do sistema Escola360.
- **Sávio Cunha:** Edição e formatação do relatório, escrevendo os tópicos visão geral, e modelo arquitetural em camadas. Escrita do componente extesionista e atualização do readme.
- **Aldemir Ferreira:** Atuou na elaboração do Modelo Arquitetural através da construção e definição dos Componentes de Aplicação.
- **João Paulo:**

V - Insira abaixo evidências das contribuições coletivas (fotos de reuniões, etc) e individuais (que demonstrem o que cada membro fez).

Histórico de versões	Histórico de versões	Histórico de versões
Todas as versões Hoje > 27 de junho, 21:00 - Versão atual ● FRANCISCO DIOGO DE SOUSA SILVA ● FRANCISCO SAVIO SOUSA DA CUNHA > 27 de junho, 18:11 ● BEATRIZ BENIGNO DE VASCONCELOS > 27 de junho, 16:37 ● FRANCISCO SAVIO SOUSA DA CUNHA ● BEATRIZ BENIGNO DE VASCONCELOS > 27 de junho, 13:44 ● FRANCISCO SAVIO SOUSA DA CUNHA 27 de junho, 11:21 ● Todos os usuários anônimos > 27 de junho, 00:18 ● FRANCISCO DIOGO DE SOUSA SILVA ● BEATRIZ BENIGNO DE VASCONCELOS	Todas as versões ● Todos os usuários anônimos > 26 de junho, 14:34 ● Todos os usuários anônimos > 26 de junho, 13:50 ● Todos os usuários anônimos > 26 de junho, 12:15 ● Todos os usuários anônimos 26 de junho, 07:28 ● ALDEMIR FERREIRA DA SILVA JÚNIOR > 26 de junho, 01:18 ● Todos os usuários anônimos - Quinta-feira > 25 de junho, 23:54 ● Todos os usuários anônimos	Todas as versões ● Todos os usuários anônimos - Quinta-feira > 25 de junho, 23:54 ● Todos os usuários anônimos > 25 de junho, 22:58 ● Todos os usuários anônimos > 25 de junho, 20:10 ● Todos os usuários anônimos - Quarta-feira > 24 de junho, 21:44 ● Todos os usuários anônimos - Terça-feira > 23 de junho, 23:14 ● FRANCISCO SAVIO SOUSA DA CUNHA > 23 de junho, 21:20 ● FRANCISCO SAVIO SOUSA DA CUNHA

Commits on Jun 27, 2026
Adicionar seção de Arquitetura de Software ao README saviocunha authored 3 hours ago Verified 745dc83
Update README.md beatrizbenigno-web authored 2 hours ago Verified 6c66179
Fix formatting issue in README.md beatrizbenigno-web authored 2 hours ago Verified 56e4faa
Projeto arquitetural do sistema saviocunha committed 5 minutes ago 284ad30
Update de seção do readme. saviocunha authored 1 minute ago Verified d498b4e

VI - Após enviar o presente relatório no AVA, cada membro deve responder ao [formulário de autoavaliação do time](#). Tal formulário deverá ser respondido somente após a conclusão da sprint.